0271. Encode and Decode Strings

Category: Arrays & Hashing | LeetCode: https://leetcode.com/problems/encode-a

Problem

- Design encode(List<String>) → String and decode(String) → List<String> so that decode(encode(x)) == x.
- Strings may contain any character, including delimiters you might want to use.

Intuition

- A plain delimiter fails because the delimiter can appear inside a word.
- Length-prefixing solves it: write "length#word". While decoding, read digits up to '#', then take exactly that many characters, whatever they are.

Brute Force

- Join with a rare character or escape delimiters. Escaping works but is fiddly and error-prone.

Optimal Approach

- Encode: for each s append s.length(), '#', s.
- Decode with pointer i: find j = first '#' at or after i; len = parse(i..j); word = substring(j+1, j+1+len); i = j+1+len.

Worked Example

```
Input ["lint", "co#de", ""].

encode -> "4#lint5#co#de0#"
decode i=0  read "4"  -> "lint"   i=6
       i=6  read "5"  -> "co#de"  i=13
       i=13 read "0"  -> ""       i=15 end
```

Java Solution

```java
public class EncodeAndDecodeStrings {
    public String encode(List<String> strs) {
        StringBuilder sb = new StringBuilder();
        for (String s : strs) sb.append(s.length()).append('#').append(s);
        return sb.toString();
    }

    public List<String> decode(String s) {
        List<String> out = new ArrayList<>();
        int i = 0;
        while (i < s.length()) {
            int j = s.indexOf('#', i);
            int len = Integer.parseInt(s.substring(i, j));
            out.add(s.substring(j + 1, j + 1 + len));
            i = j + 1 + len;
        }
        return out;
    }
}
```

Complexity

- Time: O(total characters) for both directions.
- Space: O(total characters) for the output.

Pitfalls

- Searching '#' from the start of the string instead of from i.
- Empty strings and an empty list must both round-trip (encode([]) = "").